FRA503 Deep Reinforcement Learning for Robotics

Homework 2: Cart Pole Model-Free Algorithms

Name	**Student ID**
Chantouch Orungrote	66340500011
Sasish Kaewsing	66340500076

1 CartPole Problem

CartPole, is a standard problem in reinforcement learning. by training an agent to apply forces to a cart moving along a track so as to keep a pole hinged to the cart from falling over.

A failure is said to occur if the pole falls past a given angle from vertical or if the cart runs off the track. The task is usually divided into two phases swing-up, where the pole is brought from a downward position to upright, and stabilization, where the agent keeps the pole balanced.

1.1 Mathematical Formulation

Concept	**Form**	**Definition**
Discounted Return	$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$	The total accumulated reward from time step t, discounted by γ, which the agent seeks to maximize in an MDP.
State-Action Value	$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$	The expected return starting from state s, taking action a, and following policy π thereafter.
TD Target	$Y_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	The estimated return used in Temporal Difference learning to update the current value estimate.
TD Error	$\delta_t = Y_t - Q(S_t, A_t)$	The difference between the TD Target and the current prediction, driving the learning process.
Off-Policy Learning	$\pi(a\|s) \neq b(a\|s)$	Learning the target policy π while acting according to a different behavior policy b.

Table 1: Mathematical formulation of Model-Free Control

1.2 Model-Free Control Algorithms

Monte Carlo Control updates the action-value function using the actual return G_t calculated at the end of an episode. It relies on averaging sample returns without bootstrapping.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big(G_t - Q(S_t, A_t)\big) \tag{1}$$

SARSA (State-Action-Reward-State-Action) is an on-policy TD control algorithm. It updates the Q-value using the current reward and the Q-value of the next state-action pair (S_{t+1}, A_{t+1}) actually chosen by the behavior policy.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big] \tag{2}$$

Q-Learning is an off-policy TD control algorithm. It approximates the optimal action-value function independent of the policy being followed. The update target uses the maximum Q-value of the next state.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big] \tag{3}$$

Double Q-Learning addresses the maximization bias issue in Q-Learning by using two separate value functions, Q_1 and Q_2. One function is used to determine the maximizing action, while the other is used to estimate its value.

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q_2\big(S_{t+1}, \arg\max_a Q_1(S_{t+1}, a)\big) - Q_1(S_t, A_t)\Big] \tag{4}$$

2 Experimental Design

In this experiment, we evaluate four model-free control algorithms: Monte Carlo, SARSA, Q-Learning, and Double Q-Learning. The agents are trained on the CartPole.

The objective is to analyze how different hyperparameter. Specifically, the resolution of the action space and the granularity of state discretization affect the learning performance and stability of each algorithm.

2.1 Hyperparameters

All agents share a common set of hyperparameters to ensure a fair comparison using an ϵ-greedy policy with linear decay for exploration strategy, and the parameters are listed in Table 2.

Parameter	**Value**
Episodes	10,000
Discount Factor (γ)	0.99
Learning Rate (α)	0.1
Initial ϵ	1.0
Final ϵ	0.01
ϵ-decay	0.001
Action Range	$[-10.0, 10.0]$

Table 2: Common hyperparameters used across all algorithms.

2.2 Experimental Variations

We evaluate the algorithms' sensitivity to the resolution of the observation and action spaces. The experiments are divided into two variations. In the second variation, we modify the state discretization weights vector, $W = [w_x, w_{\dot{x}}, w_\theta, w_{\dot{\theta}}]$, which scales the granularity for $x, \dot{x}, \theta, \dot{\theta}$, respectively.

Configuration	**Number of Actions**	**State Weights**
Variation 1: Action Space Resolution		
Low Resolution	5	$[2, 10, 1, 2]$
Normal Resolution	11	$[2, 10, 1, 2]$
High Resolution	21	$[2, 10, 1, 2]$
Variation 2: Observation Space Discretization		
Low Granularity	11	$[1, 5, 1, 1]$
Normal Granularity	11	$[2, 10, 1, 2]$
High Granularity	11	$[4, 20, 2, 4]$

Table 3: Hyperparameter settings for action and observation space discretization experiments.

2.3 Evaluation Metrics

- **Episode Return (G_t):** The cumulative reward obtained in a single episode, smoothed over a 100-episode window. This metric demonstrates the *learning speed* and *convergence stability*.
- **Q-Value Surface ($V(s) \approx \max_a Q(s, a)$):** A 3D surface plot mapping the maximum Q-values against Cart Position (x) and Pole Angle (θ). This visualizes whether the agent correctly identifies safe states versus critical failure states.

3 Analysis

3.1 Algorithm Performance Comparison

The experimental results present a compelling finding that contradicts general theoretical assumptions regarding continuous control.

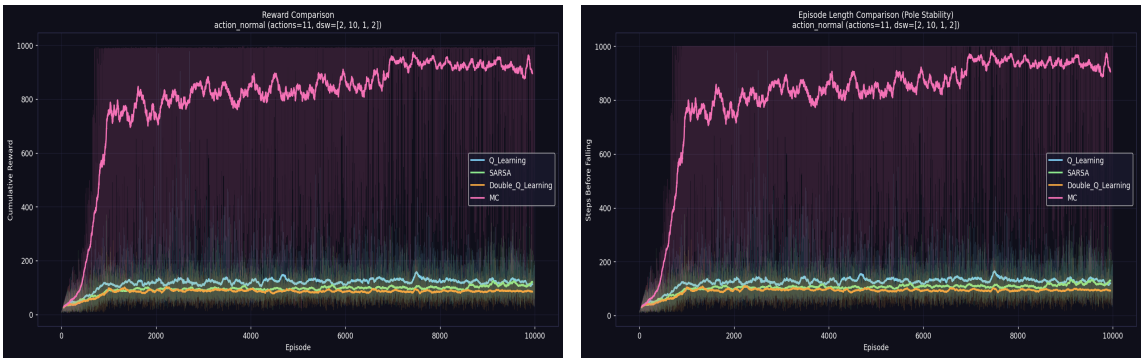

Figure 1: Comparison of action space and observation space discretization.

3.1.1 Which Algorithm Performs Best?

Dominance of Monte Carlo. The Monte Carlo method achieved the *highest average cumulative reward* and demonstrated superior stability during the deployment phase. The learning curve shows a consistent improvement that eventually surpasses the performance of TD-based methods.

Underperformance of Temporal Difference Methods. In contrast, Q-Learning, SARSA, and Double Q-Learning exhibited slower learning rates and plateaued at suboptimal reward levels. Despite their theoretical advantage of bootstrapping, they failed to generate the robust control policies observed in the Monte Carlo experiments.

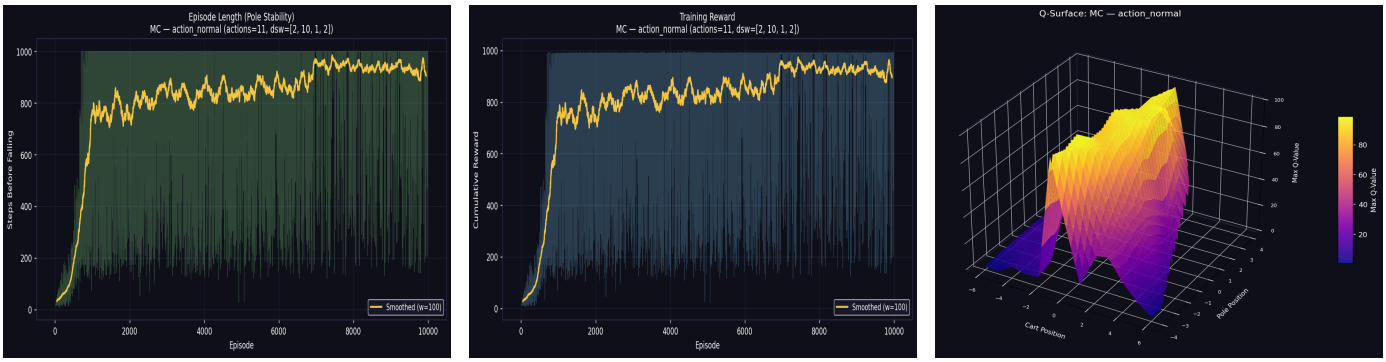

Figure 2: Monte Carlo Control performances.

3.1.2 Why Does It Perform Better than the Others?

The POMDP Challenge and Violation of Markov Property.
The discretization process fundamentally alters the nature of the environment. While the underlying physics of the CartPole system is a Markov Decision Process (MDP), the agent's perception of it through discrete bins transforms it into a **Partially Observable Markov Decision Process (POMDP)**. Because the agent cannot observe the exact continuous variables, the state transitions appear non-deterministic and the fundamental Markov property—where the future depends only on the current state is violated.

Impact of State Aliasing on Update Targets.
This partial observability manifests as **State Aliasing**, where distinct physical states are treated as identical. This creates a significant disparity between Monte Carlo and Temporal Difference (TD) methods:

- **Unbiased Convergence of Monte Carlo.** MC is naturally robust to POMDPs because it does not rely on the Markovian assumption of state transitions. By using the *actual return* (G_t) at the end of each episode, MC bypasses the corrupted intermediate state values and learns an unbiased average of the outcome. In an aliased environment, MC effectively learns the true statistical expected value of a bin.
- **Bias Propagation in TD Methods.** In contrast, Q-Learning, SARSA, and Double Q-Learning rely on *bootstrapping* ($R + \gamma Q(S')$), which assumes that the next state's value is accurate. In a POMDP, the next aliased state S' provides a flawed estimate. Since TD methods recursively update their values based on these inaccurate neighbors, the discretization error is compounded and propagated backward. This leads to the bias-dominated failure of all TD algorithms in this setup.

Robustness through Multi-modal Q-Value Surface A distinctive feature of the Monte Carlo approach is the emergence of a *multi-modal* Q-value surface, characterized by multiple peaks of high value across the state space.

- **Discovery of Diverse Strategies.** While TD methods often converge toward a single, centralized peak representing a narrow ideal balancing point, the Monte Carlo method identifies multiple pockets of

stability. Because MC updates are based on the actual survival time, it assigns high values to any state-action pair that leads to a successful trajectory, even if that trajectory is off-center or non-traditional.

- **Resilience to Perturbations.** This multi-peak landscape suggests that the MC agent has learned redundant balancing strategies. If the cart drifts away from the center or the pole angle increases, the agent can still rely on these secondary peaks of high-value knowledge to recover. In contrast, TD agents, which focus on a single localized optimum, often struggle to maintain stability once the system enters a region where their bootstrapped estimates are sparse or biased.

Inefficiency of Double Q-Learning.
Double Q-Learning failed to provide any advantage in this context for two primary reasons. First, the algorithm is specifically designed to mitigate *maximization bias* (overestimation of values), whereas the primary challenge in this experiment was state ambiguity caused by aliasing, meaning it addressed the wrong theoretical problem. Second, by splitting the experience data into two separate Q-tables for decoupled updates, the algorithm effectively halved its **sample efficiency**. In a data-constrained environment where the agent is already struggling with a sparse and aliased state space, this reduction in learning speed caused it to perform significantly worse than standard Q-Learning.

3.2 How Do the Resolutions of the Action Space and Observation Space Affect the Learning Process? Why?

The granularity of the observation and action spaces is a fundamental design choice that dictates the agent's ability to perceive the environment and execute precise control. Our analysis reveals that these resolutions significantly influence the learning process through a trade-off between information accuracy and computational tractability.

3.2.1 Observation Space Resolution

The resolution of the observation space determines how finely the continuous physical variables are partitioned into discrete bins. This choice introduces two opposing challenges:

- **State Aliasing at Low Resolution.** When the discretization is too coarse, the agent suffers from *state aliasing*. In this regime, distinct physical configurations—such as the pole leaning at a critical angle versus a near-vertical position—are mapped to the same discrete identifier. This loss of information effectively transforms the environment into a POMDP. Because the agent cannot distinguish between these different physical states, the resulting policy becomes inconsistent, leading to failures in maintaining long-term equilibrium.
- **Curse of Dimensionality at High Resolution.** Conversely, increasing the resolution to achieve higher precision triggers the *curse of dimensionality*. For the CartPole system, the state space grows exponentially (N^4) relative to the number of bins per variable. In a tabular setting with a fixed training budget of 10,000 episodes, this results in extreme *data sparsity*. The Q-table becomes too large to be adequately explored, meaning the agent rarely revisits identical states. This prevents the value function from converging, as the agent perceives nearly identical physical situations as entirely novel and unrelated events.

3.2.2 Action Space Resolution

The discretization of the force applied to the cart similarly affects the agent's learning trajectory:

- **Coarse Action Selection:** A small action space (e.g., only maximum left or right force) reduces the complexity of the search space, allowing for rapid initial learning. However, this often results in bang-bang control behavior. While the agent can keep the pole upright, it lacks the fine motor precision required to stabilize the system at a true equilibrium, often leading to jittery or oscillatory movements.
- **Fine Action Selection:** A higher resolution in the action space provides the theoretical potential for smooth and precise stabilization. However, it significantly increases *exploration complexity*. The agent must expend more episodes to evaluate and differentiate between the utility of similar force magnitudes (e.g., 5.0N vs. 5.1N). Without sufficient samples, the agent struggles to identify the optimal force for a given state, often plateauing at a sub-optimal policy.

4 Conclusion

This experiment evaluated the performance of Tabular Reinforcement Learning algorithms—Monte Carlo, Q-Learning, SARSA, and Double Q-Learning—in the continuous CartPole control task through discretization.

4.1 Summary of Findings

Our results demonstrate that in a discretized environment, **Monte Carlo (MC)** is the superior approach. Unlike Temporal Difference (TD) methods that suffer from **bias propagation** due to bootstrapping from inaccurate, aliased state estimates, the Monte Carlo method relies on **actual returns**. This makes it inherently more robust to the loss of the Markov property caused by coarse discretization.

Furthermore, we identified a critical trade-off in discretization resolution:

- **Low Resolution** leads to **State Aliasing**, where the agent lacks sufficient information to maintain stability.
- **High Resolution** triggers the **Curse of Dimensionality**, causing a sparse Q-table that the agent cannot sufficiently explore within a limited training budget.

4.2 Fundamental Limitations and Future Directions

The primary reason for the challenges observed is the **lack of generalization** in tabular methods. A Q-table treats each discrete bin as an independent entity, unable to transfer knowledge to physically similar neighboring states.

To overcome these limitations and achieve true continuous control, future work should transition toward **Deep Reinforcement Learning (DRL)**. The following improvements are proposed:

- **Neural Function Approximation.** Replacing the Q-table with a Neural Network (e.g., Deep Q-Network) allows the agent to represent continuous state spaces without manual discretization. This provides *generalization*, where the agent can infer the value of unvisited states based on similar experiences.
- **Continuous Action Spaces.** Utilizing Policy Gradient methods, such as Deep Deterministic Policy Gradient (DDPG) or Proximal Policy Optimization (PPO), would allow the agent to apply precise, continuous force rather than being restricted to a finite set of discrete actions.

In conclusion, while Tabular RL serves as a vital foundation for understanding reinforcement learning, its reliance on discretization makes it inherently inefficient for high-dimensional, continuous robotics tasks. The transition to function approximation is necessary to achieve robust and scalable autonomous control.